

# Spring Boot — Today's Complete Interview Notes

Session: 7:00 PM – 9:00 PM

Today you learned the **fundamentals of building REST APIs in Spring Boot**, especially the **Controller → Service → Model** architecture and CRUD operations.

---

## 1. What is Spring Boot?

**Spring Boot** is a framework built on top of the Spring Framework that makes it easier to create and run Java applications, especially **web applications and REST APIs**.

### Interview answer

**Spring Boot is a Java framework built on top of Spring that simplifies application development by providing auto-configuration, embedded servers, starter dependencies, and minimal configuration.**

For example, your application runs using:

```
@SpringBootApplication
public class SpringbootTrainingApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringbootTrainingApplication.class, args);
    }
}
```

---

## 2. @SpringBootApplication

You used:

```
@SpringBootApplication
public class SpringbootTrainingApplication {
```

@SpringBootApplication is the main annotation used to start a Spring Boot application.

It combines three important annotations:

```
@SpringBootApplication  
@EnableAutoConfiguration  
@ComponentScan
```

### Interview answer

@SpringBootApplication is a combination of @SpringBootApplication, @EnableAutoConfiguration, and @ComponentScan. It marks the main configuration class and enables Spring Boot's auto-configuration and component scanning.

---

## 3. Starting the Application

You used:

```
SpringApplication.run(  
    SpringbootTrainingApplication.class  
);
```

This starts the Spring Boot application.

Spring Boot starts an embedded server, such as **Tomcat**, and your API becomes accessible through:

```
http://localhost:8080
```

You saw this in your terminal:

```
Tomcat started on port 8080
```

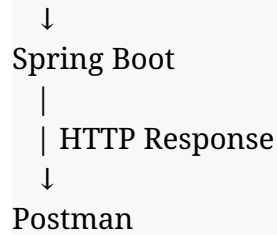
---

## 4. What is a REST API?

A REST API allows a client and server to communicate using HTTP.

For example:

```
Postman  
|  
| HTTP Request
```



A client can be:

- Browser
  - Postman
  - React application
  - Mobile application
  - Another backend
- 

## 5. @RestController

You created:

```
@RestController  
public class StaffController {
```

What does it mean?

@RestController tells Spring:

This class will handle HTTP requests and return data directly as the HTTP response.

Usually, the returned Java objects are converted to **JSON**.

For example:

```
@GetMapping("/api/staff")  
public List<Staff> getStaff() {  
    return staffService.getStaff();  
}
```

The Java:

```
List<Staff>
```

becomes JSON:

```
[
  {
    "id": 1,
    "name": "Rahul",
    "department": "Cardiology",
    "role": "Nurse"
  }
]
```

## Interview question

**Q: What is @RestController?**

**Answer:**

@RestController is used to create REST controllers in Spring Boot. It combines @Controller and @ResponseBody, so the return value of controller methods is written directly to the HTTP response body, usually as JSON.

---

## 6. Controller

Your controller is:

```
@RestController
public class StaffController {

    private final StaffService staffService;

    public StaffController(StaffService staffService) {
        this.staffService = staffService;
    }
}
```

The **Controller's responsibility** is to:

- Receive HTTP requests
- Read request data
- Call the appropriate service
- Return the response

It should **not contain all business logic**.

---

## 7. Service

You created:

```
@Service
public class StaffService {
```

The service contains the **business/application logic**.

For example:

```
public Staff getStaffById(int id) {
    for (Staff staff : staffList) {
        if (staff.getId() == id) {
            return staff;
        }
    }
    return null;
}
```

### Interview answer

The Service layer contains business logic and separates application logic from the Controller.

---

## 8. @Service

You learned that:

```
@Service
public class StaffService {
```

tells Spring that StaffService is a Spring-managed component.

Spring creates and manages its object.

Conceptually:

Without Spring:

```
StaffService service = new StaffService();
```

With Spring:

Spring creates StaffService object



Spring manages it as a Bean



Controller receives it

## Interview question

### Q: What is a Spring Bean?

A Spring Bean is an object that is created, configured, and managed by the Spring IoC container.

---

## 9. Dependency Injection

Your controller contains:

```
private final StaffService staffService;
```

and:

```
public StaffController(StaffService staffService) {  
    this.staffService = staffService;  
}
```

This is **constructor dependency injection**.

The Controller needs StaffService, so Spring provides the StaffService bean through the constructor.

Spring Container



| creates StaffService



StaffService Bean



| injects



StaffController

## Interview answer

Dependency Injection is a mechanism where Spring provides the required dependencies to a class instead of the class creating those dependencies itself.

## Why use it?

It gives:

- Loose coupling
  - Better testability
  - Cleaner code
  - Easier maintenance
- 

## 10. Model Class

You created:

```
public class Staff {  
  
    private int id;  
    private String name;  
    private String department;  
    private String role;  
}
```

This is your **Model/POJO**.

It represents a Staff object.

Example:

```
Staff staff = new Staff(  
    1,  
    "Rahul",  
    "Cardiology",  
    "Nurse"  
);
```

Conceptually:

```
Staff
├── id
├── name
├── department
└── role
```

---

## 11. Constructor

You created two constructors:

```
public Staff() {
}
```

and:

```
public Staff(
    int id,
    String name,
    String department,
    String role
) {
    this.id = id;
    this.name = name;
    this.department = department;
    this.role = role;
}
```

The parameterized constructor is useful for creating objects with data:

```
new Staff(1, "Rahul", "Cardiology", "Nurse");
```

The **no-argument constructor** is important when Spring/Jackson needs to create a Staff object from incoming JSON.

---

## 12. Getters and Setters

You generated:

```
public int getId() {
    return id;
}
```

```
}  
  
public void setId(int id) {  
    this.id = id;  
}
```

Similarly for:

```
name  
department  
role
```

## Why?

Getters retrieve values:

```
staff.getName();
```

Setters modify values:

```
staff.setName("Priya Sharma");
```

You used setters during the PUT operation.

---

## 13. GET Request

You created:

```
@GetMapping("/api/staff")  
public List<Staff> getStaff() {  
    return staffService.getStaff();  
}
```

A GET request is normally used to **retrieve data**.

Example:

```
GET /api/staff
```

Response:

```
[  
  {  
    "id": 1,  
    "name": "Rahul"  }  
]
```

```
},  
{  
  "id": 2,  
  "name": "Priya"  
}  
]
```

---

## 14. @GetMapping

```
@GetMapping("/api/staff")
```

It maps an HTTP GET request to a Java method.

### Interview answer

@GetMapping is used to map HTTP GET requests to a specific controller method.

---

## 15. POST Request

You created:

```
@PostMapping("/api/staff")  
public Staff addStaff(@RequestBody Staff staff) {  
  return staffService.addStaff(staff);  
}
```

POST is generally used to **create a new resource**.

For example, Postman sends:

```
{  
  "id": 4,  
  "name": "Sneha",  
  "department": "ICU",  
  "role": "Doctor"  
}
```

Spring converts the JSON into a Java object:

```
Staff staff
```

Then:

```
staffService.addStaff(staff);
```

adds it to the list.

---

## 16. @RequestBody

You used:

```
@RequestBody Staff staff
```

This means:

Take the data from the HTTP request body and convert it into a Staff Java object.

Flow:

JSON

↓

@RequestBody

↓

Staff object

↓

Service

For example:

```
{
  "id": 4,
  "name": "Sneha",
  "department": "ICU",
  "role": "Doctor"
}
```

becomes conceptually:

```
Staff staff = new Staff();
```

```
staff.setId(4);
```

```
staff.setName("Sneha");
```

```
staff.setDepartment("ICU");
```

```
staff.setRole("Doctor");
```

Spring/Jackson handles this conversion for you.

---

## 17. Important Correction: POST and Browser

One thing to remember for interviews:

✗ Don't say:

POST request comes from the browser URL.

That's not the correct distinction.

Both GET and POST are **HTTP methods**, and both can be sent by browsers, Postman, React, mobile apps, etc.

The difference is mainly **how the request is structured and what it is intended to do**.

For example:

### GET

```
GET /api/staff/2
```

Usually retrieves data.

### POST

```
POST /api/staff
```

with JSON body:

```
{  
  "id": 4,  
  "name": "Sneha"  
}
```

Usually creates data.

---

## 18. GET by ID

You created:

```
@GetMapping("/api/staff/{id}")
public Staff getStaffById(@PathVariable int id) {
    return staffService.getStaffById(id);
}
```

Request:

GET /api/staff/2

Spring takes:

2

and puts it into:

int id

Then service searches the list.

Result:

```
{
  "id": 2,
  "name": "Priya",
  "department": "Emergency",
  "role": "Doctor"
}
```

---

## 19. @PathVariable

@PathVariable int id

is used to extract a value from the URL path.

Example:

/api/staff/2

Here:

2 → id

## Interview answer

@PathVariable is used to extract a value from the URI path and bind it to a controller method parameter.

---

## 20. PUT Request

You created:

```
@PutMapping("/api/staff/{id}")
public Staff updateStaff(
    @PathVariable int id,
    @RequestBody Staff staff) {

    return staffService.updateStaff(id, staff);
}
```

PUT is generally used to **update an existing resource**.

Postman request:

PUT /api/staff/2

Body:

```
{
  "id": 2,
  "name": "Priya Sharma",
  "department": "Emergency",
  "role": "Senior Doctor"
}
```

Your service finds ID 2:

```
if (staff2.getId() == id)
```

Then updates:

```
staff2.setName(staff.getName());
staff2.setDepartment(staff.getDepartment());
staff2.setRole(staff.getRole());
```

---

## 21. Why do we use both `@PathVariable` and `@RequestBody` in PUT?

This is an **important interview question**.

Request:

```
PUT /api/staff/2
```

The:

```
2
```

identifies **which staff to update**.

The JSON body contains:

```
{
  "id": 2,
  "name": "Priya Sharma",
  "department": "Emergency",
  "role": "Senior Doctor"
}
```

which contains **the new data**.

So:

`@PathVariable` → Which staff?

`@RequestBody` → What new data?

---

## 22. DELETE Request

You created:

```
@DeleteMapping("/api/staff/{id}")
public String deleteStaff(@PathVariable int id) {
    return staffService.deleteStaff(id);
}
```

Example:

```
DELETE /api/staff/3
```

Your service:

```
boolean removed =  
    staffList.removeIf(staff -> staff.getId() == id);
```

If the staff exists:

Staff deleted successfully

Otherwise:

Staff not found

---

## 23. removeIf()

You learned:

```
staffList.removeIf(  
    staff -> staff.getId() == id  
);
```

The lambda:

```
staff -> staff.getId() == id
```

checks every Staff object.

If the condition is true, the object is removed.

Example:

```
Staff 1 → false  
Staff 2 → true ← remove  
Staff 3 → false
```

removeIf() returns:

```
true → at least one object was removed  
false → nothing was removed
```

---

## 24. Request Parameters

You learned @RequestParam.

A query parameter looks like:

```
/api/staff/department?department=Cardiology
```

Here:

```
department=Cardiology
```

is a query parameter.

Controller:

```
@GetMapping("/api/staff/department")
public List<Staff> getStaffsByDepartment(
    @RequestParam String department) {

    return staffService.getStaffsByDepartment(department);
}
```

Spring gets:

```
Cardiology
```

and puts it into:

```
String department
```

---

## 25. @RequestParam VS @PathVariable

Very important for interviews.

### PathVariable

```
/api/staff/2
```

```
@PathVariable int id
```

Used when the value is part of the URL path and identifies a resource.

### RequestParam

```
/api/staff/department?department=Cardiology
```

`@RequestParam` String department

Used for query parameters, often for filtering/searching/sorting/pagination.

### Easy way to remember

PathVariable

↓

/staff/2

RequestParam

↓

/staff?department=Cardiology

---

## 26. Department Filtering

Your service:

```
public List<Staff> getStaffsByDepartment(String department) {
```

```
    List<Staff> staffByDepartment = new ArrayList<>();
```

```
    for (Staff staff3 : staffList) {
```

```
        if (staff3.getDepartment()  
            .equalsIgnoreCase(department)) {
```

```
            staffByDepartment.add(staff3);
```

```
        }
```

```
    }
```

```
    return staffByDepartment;
```

```
}
```

If you send:

GET /api/staff/department?department=Cardiology

you get:

```
[
```

```
{
```

```
"id": 1,  
"name": "Rahul",  
"department": "Cardiology",  
"role": "Nurse"  
}  
]
```

You also learned:

`equalsIgnoreCase()`

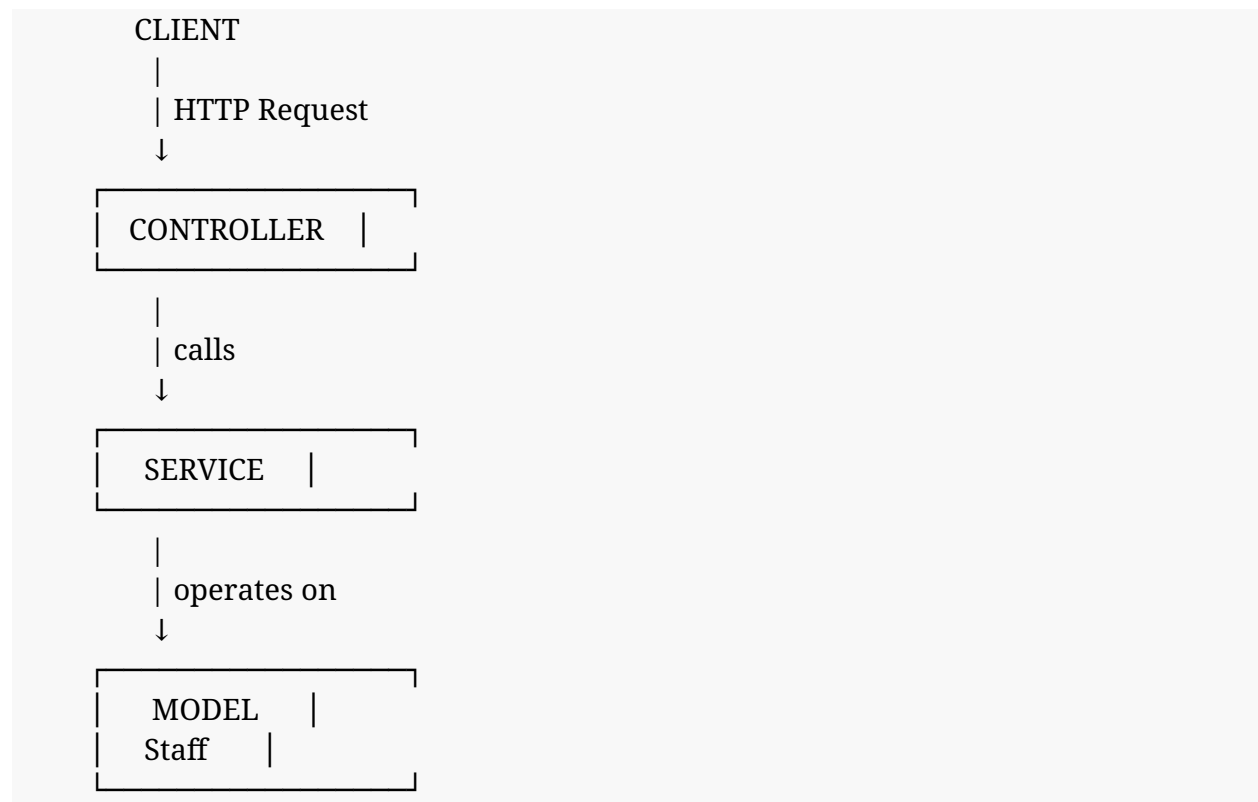
which compares strings without considering uppercase/lowercase differences.

---

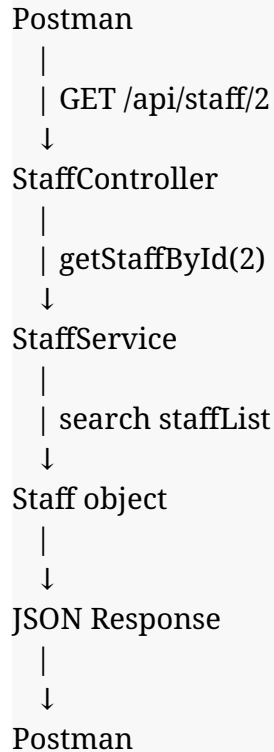
## 27. Your Complete Architecture

This is one of the **most important things to understand for interviews**.

Your application follows:



For example:



## 28. Why Controller → Service?

Suppose you put everything inside Controller:

```
@GetMapping("/api/staff/{id}")  
public Staff getStaffById(@PathVariable int id) {  
  
    // lots of business logic here  
}
```

Your controller becomes difficult to maintain.

Instead:

```
Controller  
↓  
Service
```

Controller handles HTTP-related work.

Service handles business logic.

## Interview answer

We separate Controller and Service responsibilities to achieve separation of concerns, improve maintainability, testability, and keep the code loosely coupled.

---

## 29. Your CRUD API

You have now built a basic CRUD REST API.

### Create

```
POST /api/staff
```

### Read all

```
GET /api/staff
```

### Read one

```
GET /api/staff/{id}
```

### Update

```
PUT /api/staff/{id}
```

### Delete

```
DELETE /api/staff/{id}
```

So:

```
C → POST
```

```
R → GET
```

```
U → PUT
```

```
D → DELETE
```

**Remember this for interviews.**

---

## 30. Postman

You learned to use **Postman** to test your REST APIs.

Postman allows you to send HTTP requests without needing to build a frontend.

For example:

```
GET  
http://localhost:8080/api/staff
```

or:

```
POST  
http://localhost:8080/api/staff
```

with JSON body.

You used Postman to successfully test:

- GET
  - POST
  - PUT
  - Query parameter filtering
- 

## 31. HTTP Status Codes You Encountered

200 OK

Request was successful.

You saw:

```
200 OK
```

after successfully updating a staff member.

404 Not Found

You initially saw Spring Boot's:

```
Whitelabel Error Page  
status=404
```

This means the requested endpoint/resource was not found.

405 Method Not Allowed

You saw:

```
405 Method Not Allowed
```

This happened when you sent:

```
POST /api/staff/2
```

but your application only had a GET mapping for that URL.

You then changed it to:

```
PUT /api/staff/2
```

and got:

```
200 OK
```

500 Internal Server Error

You also encountered:

```
500 Internal Server Error
```

This means an exception/error occurred on the server while processing the request.

---

## 32. localhost:8080

You used:

```
http://localhost:8080
```

localhost

Means your own computer.

8080

The port where your Spring Boot application was running.

So:

```
localhost:8080
```

means:

Access the application running on port 8080 on my computer.

---

## 33. Complete Code Structure You Built

Your project is approximately:

```
springboot-training
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.example.springboot_training
│   │   │       ├── SpringbootTrainingApplication.java
│   │   │       ├── controller
│   │   │       │   └── StaffController.java
│   │   │       ├── service
│   │   │       │   └── StaffService.java
│   │   │       └── model
│   │   │           └── Staff.java
│   └── pom.xml
```

This is a basic but proper layered structure.

---

## 34. All Annotations You Learned

| Annotation             | Purpose                                  |
|------------------------|------------------------------------------|
| @SpringBootApplication | Main Spring Boot configuration/startup   |
| @RestController        | Creates REST controller                  |
| @Service               | Creates Spring-managed service component |
| @GetMapping            | Handles GET                              |
| @PostMapping           | Handles POST                             |
| @PutMapping            | Handles PUT                              |

| Annotation     | Purpose                                   |
|----------------|-------------------------------------------|
| @DeleteMapping | Handles DELETE                            |
| @RequestBody   | Converts request body JSON to Java object |
| @PathVariable  | Gets value from URL path                  |
| @RequestParam  | Gets query parameter                      |

## 35. Interview Questions You Should Be Able to Answer

### Q1. What is Spring Boot?

Spring Boot is a Java framework built on Spring that simplifies application development through auto-configuration, starter dependencies, and embedded servers.

### Q2. What is @RestController?

It is used to create REST controllers and return data directly in the HTTP response, usually as JSON.

### Q3. What is @Service?

It marks a class as a Spring-managed service component where business logic is generally implemented.

### Q4. What is dependency injection?

Dependency Injection means Spring provides required objects/dependencies to a class instead of the class creating them itself.

### Q5. Why use constructor injection?

It makes dependencies explicit, supports immutability with final, and improves testability.

### Q6. What is @RequestBody?

It binds the HTTP request body to a Java object.

### Q7. What is @PathVariable?

It extracts a value from the URL path.

### Q8. What is @RequestParam?

It extracts a query parameter from the URL.

### Q9. Difference between PathVariable and RequestParam?

@PathVariable extracts values from the URI path, while @RequestParam extracts query parameters.

Example:

```
/api/staff/2
```

vs.

```
/api/staff/department?department=Cardiology
```

### Q10. Why shouldn't business logic be placed in Controller?

To maintain separation of concerns. Controllers handle HTTP requests while services handle business logic.

### Q11. What is CRUD?

CRUD stands for Create, Read, Update, and Delete.

Create → POST

Read → GET

Update → PUT

Delete → DELETE

### Q12. Why does Spring convert Java objects into JSON?

Spring's HTTP message conversion mechanism, commonly using Jackson for JSON, serializes Java objects into JSON responses.

### Q13. Why did you create a no-argument constructor in Staff?

It allows frameworks such as Jackson to instantiate the object when mapping JSON request data to the Java class.

---

## 36. Your Most Important Mental Model

Don't memorize Spring Boot as individual annotations.

Understand this:

```
HTTP REQUEST
↓
@RestController
↓
Controller
↓
@Service
↓
Business Logic
↓
Model
↓
HTTP RESPONSE
↓
JSON
```

And for incoming POST/PUT data:

```
JSON
↓
@RequestBody
↓
Staff Java Object
↓
Controller
↓
Service
↓
List
```

For URL parameters:

```
/api/staff/2
↓
@PathVariable
↓
int id
```

For query parameters:

?department=Cardiology

↓

@RequestParam

↓

String department

---

## 37. One Important Improvement for Your Current Project

Your current project stores data in:

```
private final List<Staff> staffList = new ArrayList<>();
```

This is **only in-memory storage**.

So if you stop/restart the Spring Boot application, the data you added through POST will disappear and your constructor will recreate the original three staff members.

That's completely fine for **today's learning**.

Later, you'll replace:

ArrayList

with:

MySQL

↓

JPA/Hibernate

↓

Repository

↓

Service

↓

Controller

That will turn your practice API into a much more realistic Spring Boot application.

---

# Today's Interview Revision — 10 Things to Memorize

If you have only **5 minutes before an interview**, remember these:

## 1. @SpringBootApplication

→ Starts/configures Spring Boot application

## 2. @RestController

→ Handles REST HTTP requests

## 3. @Service

→ Contains business logic

## 4. Dependency Injection

→ Spring provides required objects

## 5. @GetMapping

→ GET / read

## 6. @PostMapping + @RequestBody

→ POST / create + JSON → Java object

## 7. @PutMapping + @PathVariable + @RequestBody

→ Update specific resource

## 8. @DeleteMapping + @PathVariable

→ Delete specific resource

## 9. @RequestParam

→ Query parameter/filter

10. Controller → Service → Model

→ Separation of responsibilities

## Your current level after today's session

**Spring Boot fundamentals:** ● Good

**REST API basics:** ● Good

**CRUD:** ● Good

**Controller/Service understanding:** ● Good

**Postman API testing:** ● Good

**Database/JPA/Hibernate:** ⌚ Not covered yet

So today's session was a **solid Spring Boot REST fundamentals session**. The next major step should be moving from your temporary `ArrayList<Staff>` to **MySQL + JPA/Hibernate + Repository**, because that's where these concepts start becoming useful in a real backend project.